

نام و نام خانوادگی : حدیث عبودی

نام استاد مربوطه : سرکار خانم خواست خدایی

موضوع : کنترلر شارژر خورشیدی

پروژه نهایی سخت افزار

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define PANEL_PIN A0      // ولتاژ پنل خورشیدی
#define BATTERY_PIN A1    // ولتاژ باتری
#define MOSFET_PIN 9      // کنترل ماسفت (PWM)
#define LED_RED 7         // نشانگر شارژ فعال
#define LED_GREEN 8       // نشانگر شارژ کامل
#define LED_YELLOW 6      // نشانگر خطا/شب

const float BULK_VOLTAGE = 14.4;
const float FLOAT_VOLTAGE = 13.8;
const float LOW_BATTERY_CUT = 11.8;
const float VOLTAGE_DIVIDER_RATIO = (22.0 + 4.7) / 4.7;

float panelVoltage = 0;
float batteryVoltage = 0;
int batteryPercent = 0;
int chargeState = 0;
```

```
int pwmDuty = 150;
unsigned long lastDisplayTime = 0;

int calculateBatteryPercent(float voltage) {
    if (voltage >= BULK_VOLTAGE) return 100;
    if (voltage <= LOW_BATTERY_CUT) return 0;
    return map(voltage * 100, LOW_BATTERY_CUT * 100, BULK_VOLTAGE *
100, 0, 100);
}

float readVoltage(int pin) {
    float raw = analogRead(pin);
    return (raw / 1023.0) * 5.0 * VOLTAGE_DIVIDER_RATIO;
}

void drawProgressBar(int percent, int row) {
    lcd.setCursor(0, row);
    int bars = map(percent, 0, 100, 0, 16);
    for (int i = 0; i < 16; i++) {
        if (i < bars) {
            lcd.print((char)255);
        } else {
            lcd.print(" ");
        }
    }
}
```

```
void controlCharging() {  
    if (panelVoltage < 1.0 && batteryVoltage < 1.0) {  
        pwmDuty = 128;  
        analogWrite(MOSFET_PIN, pwmDuty);  
        return;  
    }  
  
    if (panelVoltage <= batteryVoltage + 0.5) {  
        pwmDuty = 0;  
        analogWrite(MOSFET_PIN, pwmDuty);  
        return;  
    }  
  
    switch (chargeState) {  
        case 0: // Bulk Charge  
            pwmDuty = 255;  
            if (batteryVoltage >= BULK_VOLTAGE) {  
                chargeState = 1;  
            }  
            break;  
  
        case 1: // Absorption  
            float targetVolt = constrain(batteryVoltage, BULK_VOLTAGE -  
0.5, BULK_VOLTAGE);  
            pwmDuty = map(targetVolt * 100, (BULK_VOLTAGE - 0.5) * 100,  
BULK_VOLTAGE * 100, 255, 120);
```

```

        if (batteryVoltage >= BULK_VOLTAGE) {
            chargeState = 2;
        }
        break;

    case 2: // Float
        if (batteryVoltage < FLOAT_VOLTAGE - 0.3) {
            pwmDuty = 180;
        } else {
            pwmDuty = 80;
        }
        break;
}

analogWrite(MOSFET_PIN, pwmDuty);
}

void updateStatusLEDs() {
    if (batteryPercent >= 100) {
        digitalWrite(LED_GREEN, HIGH);
        digitalWrite(LED_RED, LOW);
        digitalWrite(LED_YELLOW, LOW);
    }
    else if (pwmDuty > 10) {
        digitalWrite(LED_RED, HIGH);
        digitalWrite(LED_GREEN, LOW);
        digitalWrite(LED_YELLOW, LOW);
    }
}

```

```
}  
  
else if (panelVoltage <= batteryVoltage + 0.5) {  
    digitalWrite(LED_YELLOW, HIGH);  
    digitalWrite(LED_RED, LOW);  
    digitalWrite(LED_GREEN, LOW);  
}  
}  
  
void updateDisplay() {  
    lcd.clear();  
    lcd.setCursor(0, 0);  
  
    if (chargeState == 0) lcd.print("Bulk");  
    else if (chargeState == 1) lcd.print("Abs");  
    else lcd.print("Float");  
    lcd.print(" ");  
    lcd.print(batteryVoltage, 1);  
    lcd.print("V ");  
    int pwmPercent = map(pwmDuty, 0, 255, 0, 100);  
    lcd.print(pwmPercent);  
    lcd.print("%");  
    lcd.setCursor(0, 1);  
    lcd.print(batteryPercent);  
    lcd.print("% ");  
    drawProgressBar(batteryPercent, 1);  
}
```

```
void sendToSerial() {  
    Serial.print("P:");  
    Serial.print(panelVoltage, 1);  
    Serial.print("V | B:");  
    Serial.print(batteryVoltage, 1);  
    Serial.print("V | ");  
  
    if (chargeState == 0) Serial.print("Bulk");  
    else if (chargeState == 1) Serial.print("Abs");  
    else Serial.print("Float");  
  
    Serial.print(" | PWM:");  
    Serial.print(pwmDuty);  
    Serial.print(" | ");  
    Serial.print(batteryPercent);  
    Serial.println("%");  
}  
  
void setup() {  
    Serial.begin(9600);  
  
    pinMode(MOSFET_PIN, OUTPUT);  
    pinMode(LED_RED, OUTPUT);  
    pinMode(LED_GREEN, OUTPUT);  
    pinMode(LED_YELLOW, OUTPUT);  
  
    analogWrite(MOSFET_PIN, pwmDuty);  
}
```

```
    lcd.init();
    lcd.backlight();
    lcd.print("Solar Charger");
    lcd.setCursor(0, 1);
    lcd.print("Starting...");
    delay(1000);
    lcd.clear();
}

void loop() {
    panelVoltage = readVoltage(PANEL_PIN);
    batteryVoltage = readVoltage(BATTERY_PIN);
    batteryPercent = calculateBatteryPercent(batteryVoltage);

    controlCharging();
    updateStatusLEDs();

    // به روزرسانی خروجی‌ها بدون وقفه سنگین
    if (millis() - lastDisplayTime >= 500) {
        updateDisplay();
        sendToSerial();
        lastDisplayTime = millis();
    }
    delay(20);
}
```

اجرا در Proteus

